

# LSTM の初期状態の学習による DRQN の改善

Oh Hyunwoo<sup>1,a)</sup> 金子 知適<sup>2,3,b)</sup>

**概要：**強化学習は、試行錯誤から学習する点で様々な問題に適用しやすいため、近年盛んに研究されている。Deep Q-Network (DQN) は、ディープラーニングを用いた強化学習のモデルで、ゲーム画面を入力した Atari 2600 の様々なゲームに対して人間より良い性能を実現した。しかし、DQN の入力は一エジェントが観察した最近の 4 つのフレームのみを用いるため、意味がある行動をするために 4 ステップより昔の情報を必要とするような環境に対しては性能が低下する可能性がある。この欠点を解決するために Deep Recurrent Q-Network (DRQN) が提案されている。DRQN は、DQN に循環神経網の一種である LSTM を組み合わせることで過去の情報を扱う。しかし、DRQN は学習の時に LSTM の初期状態をゼロベクトルで初期化するため、学習時に用いた情報のタイムステップより長い期間に対して学習することが難しいという欠点がある。本研究では、DRQN の性能の改善を目標に、エピソードの断片を学習する際に LSTM の初期状態を適切に生成することによって時間変化に関する学習効率を高める。評価は、Atari 2600 の一部のゲームに対して従来手法と提案手法の性能を比較することで行う。

## Enhancement of DRQN by Training Initial State of LSTM

OH HYUNWOO<sup>1,a)</sup> TOMOYUKI KANEKO<sup>2,3,b)</sup>

**Abstract:** Reinforcement Learning is a kind of machine learning through agents' interaction with the environment. Deep Q-Network (DQN), which is a model of reinforcement learning empowered by deep neural networks, succeeded in learning human-level control policies on various kinds of Atari 2600 games with image pixel inputs. Because an input of DQN consists of the game frames of the last four steps, DQN had difficulty on mastering such games that need to remember events earlier than four steps in the past. To alleviate this problem, Deep Recurrent Q-Network (DRQN) was proposed. In DRQN, the first fully-connected layer just after convolutional layers is replaced with an LSTM to handle past information. The hidden state of DRQN is set to zero vector at the beginning of each update. Making the hidden state a zero vector makes it harder to learn the policy over a longer time period than the number of time steps DRQN used for the update. In this paper, we propose a new learning method of DRQN by adding a fully connected network which makes hidden state at the start of each update. Experiments of our method on six games of Atari 2600 show a level of performance between the existing methods and our method.

### 1. はじめに

強化学習はある環境で行動を繰り返しながら試行錯誤す

ることによって賢い行動政策を学ぶ学習手法である。最近のディープラーニング技術の発展と共に強化学習の研究も盛んに研究されている。特に文献 [1] で発表された Deep Q-Network (DQN) は、ゲーム画面の pixel 情報、ゲームスコアを入力として用いることで、DQN 以前の全てのアルゴリズムより良い性能を実現した。さらに、Atari 2600 の一部のゲームでは人間より良い性能を実現した。

しかし、DQN は入力としてエジェントが観察した最近の 4 つのフレームを用いるため、意味がある行動を選ぶために 4 ステップ以上の情報が必要である環境に対し

<sup>1</sup> 東京大学大学院 情報学環・学際情報学府  
Graduate School of Interdisciplinary Information Studies,  
the University of Tokyo

<sup>2</sup> 東京大学大学院情報学環  
Interfaculty Initiative in Information Studies, the University  
of Tokyo

<sup>3</sup> 国立研究開発法人科学技術振興機構 さきがけ  
JST, PRESTO

<sup>a)</sup> hyunwoo-oh@g.ecc.u-tokyo.ac.jp

<sup>b)</sup> kaneko@acm.org

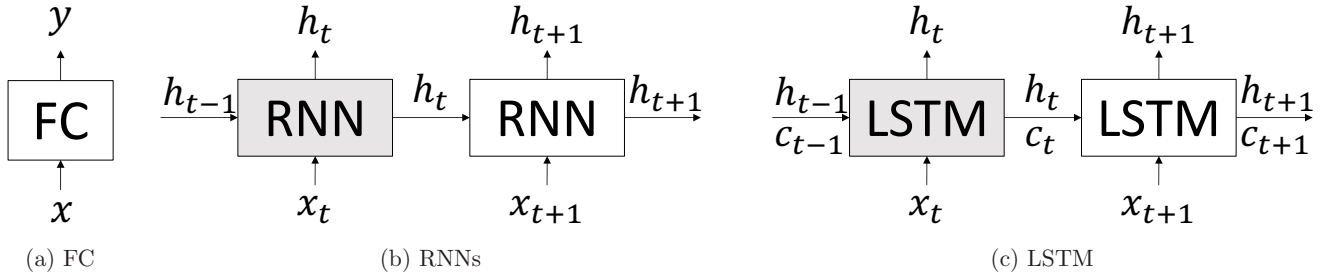


図 1: Typical architecture of FC, RNNs, and LSTM.

て性能の低下の可能性がある．この欠点を解決するために Deep Recurrent Q-Network (DRQN) が提案されている [2]．DRQN は，DQN に循環神経網 (Recurrent Neural Network, RNN) の一種である LSTM を組み合わせることで過去の情報を扱うようにした．文献 [2] では，Atari 2600 のゲームの 1 つである Pong の画面を確率  $p = 0.5$  で隠すことによって部分観察問題にし，DRQN と DQN の性能を比較した．その結果，DRQN が DQN より良い性能を実現した．しかし，学習の時に LSTM の初期状態をゼロベクトルで初期化し，特定な長さのステップの情報をを用いて学習するため，長時間の情報を学習することが難しいという欠点がある．これを解決するために文献 [3] では学習時に特定な長さのステップを全部使わず，一部のみを用いてエージェントを学習させた．この手法は簡単に既存手法よりは良い性能ができたが，ランダムサンプリングの観点で適しているとは言えない．

本研究では，DRQN の長時間の情報を学習することが難しいという欠点を改善するために，全結合ネットワーク (Fully Connected Network, FC) を用いて DRQN の循環神経網の初期入力を設定することによって DRQN の性能を高める．また，Atari 2600 の一部のゲームを対して従来の手法と提案手法の性能を比較する．

## 2. 背景

### 2.1 Recurrent Neural Networks

FC と Convolutional Neural Network (CNN) は時系列でないデータに対して良い性能を実現したが，時間的な情報を必要とする時系列データに対しては過去の情報を扱わないため，限界があった．これを解決するために，分析する対象と以前ステップの出力の 2 つを入力とすることによってネットワークが過去の情報を考慮するようにした，循環神経網 (Recurrent Neural Networks, RNNs) が提案されている．FC の入力は， $x$ ，出力  $y$ ，Weight matrix  $W$ ，bias  $b$  に対して，

$$y = Wx + b$$

と構成されている．RNNs はステップ  $t$  の入力  $x_t$ ，出力  $h_t$  に対して，

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

と構成されている．ここで  $\tanh$  は活性化関数の一種の  $\tanh$  関数である．

図 1(a) のように，FC の入力は分析する対象  $x$  の 1 つであるが，循環神経網の入力は  $x_t$ ， $h_{t-1}$  である． $x_t$  は分析する対象であり， $h_{t-1}$  は 1 ステップ前の出力で，hidden state である．LSTM [4] は，循環神経網の一種である．図 1(b) に表現されている循環神経網は，ステップの数が長くなると過去の情報を忘れる欠点がある．これを解決するために，過去の情報を覚える役割がネットワーク構造に組み込まれている．図 1 のように LSTM は  $x_t$ ， $h_{t-1}$ ， $c_{t-1}$  を入力として用いる． $x_t$  は分析する対象， $h_{t-1}$  は 1 ステップ前の出力で，hidden state である． $c_{t-1}$  は過去の情報を覚える役割をする部分で，cell state である．その後， $h_t$ ， $c_t$  を出力する． $h_t$  は現在のステップの出力にして hidden state， $c_t$  は現在のステップの cell state である．ここで，hidden state と cell state を合わせて LSTM の状態と呼び，エピソードを始める時の状態を初期状態と呼ぶ．cell state の投入で LSTM は大幅の性能の改善ができ，循環神経網の中で最もよく使われている．LSTM の式は以下のように構成されている．

$$f_t = \text{sigmoid}(W_{xh\_f}x_t + W_{hh\_f}h_{t-1} + b_{h\_f})$$

$$i_t = \text{sigmoid}(W_{xh\_i}x_t + W_{hh\_i}h_{t-1} + b_{h\_i})$$

$$o_t = \text{sigmoid}(W_{xh\_o}x_t + W_{hh\_o}h_{t-1} + b_{h\_o})$$

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t$$

$$h_t = o_t \odot (\tanh)(c_t)$$

ここで  $\text{sigmoid}$  は活性化関数の一種の  $\text{sigmoid}$  関数で， $\odot$  は要素別の掛け算を意味する．

### 2.2 Markov Decision Process

Markov Decision Process (MDP) でモデル化した環境は 4 つの要素の tuple,  $(S, A, R, P)$  で定義される．全てのステップ  $t$  で，エージェントは環境の状態  $S_t \in S$  と相互作用して行動  $A_t \in A$  を選ぶ．エージェントの行動  $A_t$  によって環境は報酬  $R_t \in R$  をリターンし，エージェントは新しい状態  $S_{t+1} \in S$  を観察する．強化学習の目的は，累積報酬を最大化する行動政策を学習することである．

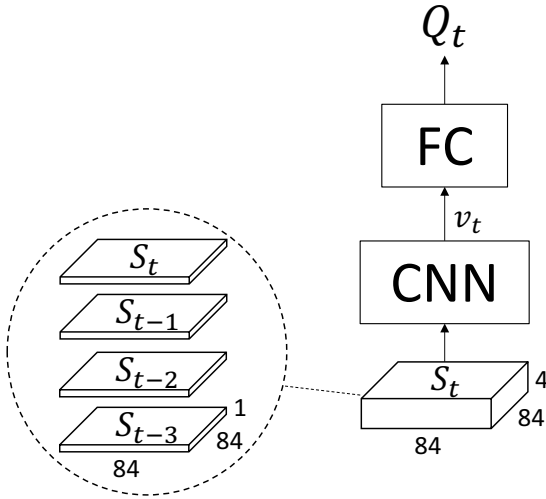


図 2: The network architecture of DQN.

### 2.3 Experience Replay

エージェントは効率良く学習するためには、時間的に隣接した特定な場面だけではなく、多様な場面で学習する必要がある。エージェントの探索と学習を行う時、一つのエピソードで経験した各ステップの状態はお互いに似ているため、経験を順に再生すると偏りが生じる。そこでランダムに選んだエピソードからランダムな時刻の経験を再生して学習することが行われる。この手法を本稿では random sampling と呼ぶ。文献 [1] で発表された DQN では、探検する時に、エージェントはエピソードで体験した各時刻の情報をリプレイメモリに保存する。ここで、情報を保持するメモリをリプレイメモリと呼ぶ。学習する時にリプレイメモリから事前に指定したサイズのサンプルをランダムにとって学習に用いた。この手法は random sampling によってエージェントが多様な場面から学習ができるという長所を持ち、最も一般的な手法として使われている。

### 2.4 Deep Q-Network

Q-関数は累積報酬の最大値を意味する Q-値を推定する。ある状態  $S_t$  で、行動政策  $\pi$  による行動  $A_t$  からの Q-値は、

$$Q_{\pi}(S_t, A_t) = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} \cdots \quad (1)$$

であり、 $\gamma \in [0, 1]$  は即時の報酬と未来の報酬の重要性に関するトレードオフを表す discount factor である。さらに最適行動方策  $\pi^*$  によって、式 (1) は以下のように変形することができる。

$$Q_{\pi^*}(S_t, A_t) = R_{t+1} + \gamma \max_{a'} Q_{\pi^*}(S_{t+1}, a') \quad (2)$$

エージェントが推定した Q-値によって式 (2) の左辺と右辺が同じ値になると、エージェントは Q-値を完璧に推定したと言える。つまり、エージェントは式 (2) の左辺と右辺が一致するような方向に学習を行い、行動政策を学習する。文献 [1] で発表された DQN ではニューラルネットワーク

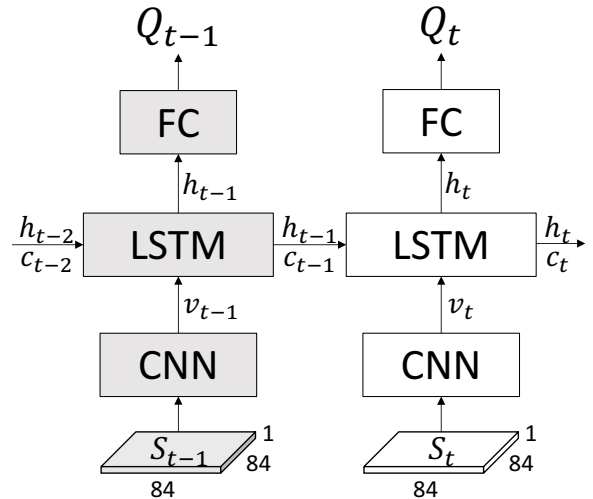


図 3: The network architectures of DRQN.

を用いて Q-関数を表現し、その損失関数は式 (3) である。

$$\left( R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a' | \theta^-) - Q(S_t, A_t | \theta) \right)^2, \quad (3)$$

損失関数の  $\theta$  はエージェントのネットワークのパラメータであり、エージェントの行動政策を意味する。学習時に DQN はステップ  $t+1$  の時の Q-値を利用するために、事前に指定した期間の前のエージェントのパラメータに固定された target network を用いた。DQN は毎ステップに学習を行うため、学習の目的となる行動政策もステップ毎に変化するため、target network がないとエージェントは着実に学習することが難しい。Target network を用いることで DQN の性能が大幅に改善された [1]。

図 2 は DQN のネットワーク構造を示す。現在のステップを  $t$  とする時、入力ステップ  $t-3, t-2, t-1, t$  のゲームフレームとなる。元々は RGB の 3 チャンネルだった各フレームを灰色の 1 チャンネルに変換して、図 2 のようにフレーム 1 つ 1 つをチャンネルとする 4 チャンネルの入力として扱う。特徴  $v_t$  は入力から CNN によって抽出される。最後に Fully Connected Network (FC) は特徴  $v_t$  を用いて Q-値  $Q_t$  を抽出する。

## 3. 先行研究

### 3.1 Deep Recurrent Q-Network

DQN は入力としてエージェントが観察した最近の 4 つのフレームを用いるため、意味がある行動を選ぶために 4 ステップ以上の情報が必要である環境に対して性能の低下の可能性がある。実際に文献 [1] の実験によると、Atari 2600 に対して一部のゲームでは人間より高いスコアを実現したが、学習が全然できなかったゲームもあった。この欠点を解決するために文献 [2] では Deep Recurrent Q-Network (DRQN) が発表された。DRQN は、図 3 のように CNN 層と最後の FC 層の間に循環神経網の一種である LSTM 層を追加したものである。これにより過去が Q-

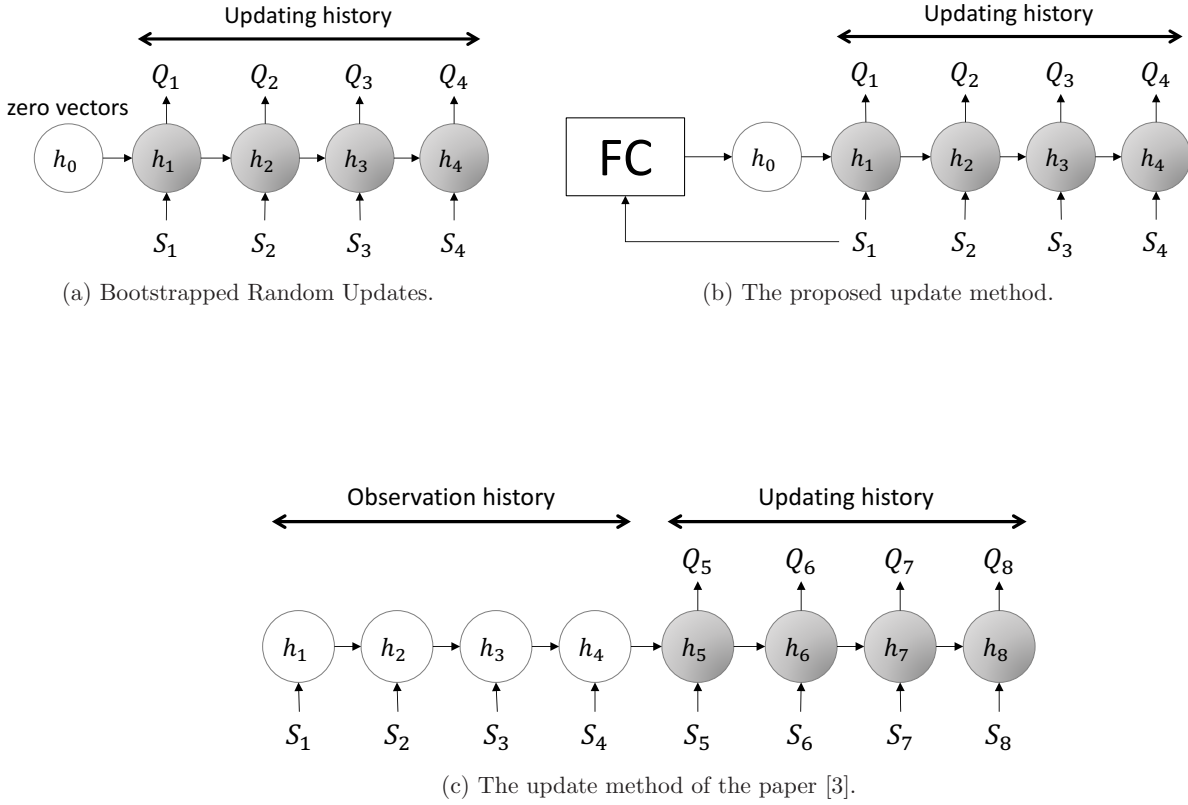


図 4: Update methods of DRQN.

値に反映されるようになった。DRQN の入力は、灰色に変換された現在ステップ  $t$  のゲームフレームと、以前ステップ  $t-1$  の hidden state  $h_{t-1}$  と memory state  $c_{t-1}$  である。DRQN は学習する時に特別な工夫を用いて LSTM 層が過去の情報を考慮するようにした。以下にその学習方法の詳細を文献 [2] で示された分類に従って述べる。

### 3.1.1 Bootstrapped Sequential Updates

Random sampling は、リプレイメモリからランダムにエピソードを選び、各エピソードの中で事前に指定した長さのステップで情報をとって学習を行う。Bootstrapped Sequential Updates は、Random sampling とは違い、リプレイメモリからランダムに選んだエピソード全体の情報を用いて学習を行う。LSTM 層の hidden state は学習の初期段階にゼロベクトルで初期化され、時刻に沿って観察と行動を再生しながら学習する際に、hidden state は過去の情報を保存する役割を果たす。この方法はエピソードの最初から最後までを LSTM 層の hidden state に流せるため、LSTM 層が時間的に連続している情報を学習できるという長所があるが、多様な場面を学習するための random sampling の観点では適しているとは言えない。

### 3.1.2 Bootstrapped Random Updates

学習する時に、random sampling のようにリプレイメモリからランダムに選ばれたエピソードの中で、事前に指定したサイズのサンプルをランダムにとって学習に用いる。図 4(a) は、Bootstrapped Random Updates がアップデー

トする時にリプレイメモリの要素を使う様子である。 $S_1, S_2, S_3, S_4$  は、リプレイメモリからのランダムに選んだエピソードの中で、サイズを 4 にしたサンプルである。 $S_t$  を入力して  $Q_t$  が抽出され、 $Q_1, Q_2, Q_3, Q_4$  を用いてエージェントを学習させる。ここで、アップデートに用いるサンプルを Updating history と呼ぶ。Bootstrapped Random Updates は学習の初期段階に LSTM の初期状態、 $h_0$  をゼロベクトルにする。Bootstrapped Sequential Updates より学習データの多様性に適する方法だと言えるが、LSTM 層の初期状態は学習の初期段階にゼロベクトルで初期化されるため、LSTM 層が長い期間の情報を学習することは難しいという欠点がある。

文献 [2] では、部分観察問題ではなかった Atari2600 に、0.5 の確率でゲームの画面を隠すトリックを適用することで、部分観察問題に変更した。Bootstrapped Sequential Updates と Bootstrapped Random Updates で実験した結果、両手法の性能が似ていることが報告された。

### 3.2 Sequential Updates

文献 [3] で発表された手法は、Bootstrapped Random Updates の長時間の情報を学習することが難しいという欠点を解決するために、学習時に簡単なトリックを用いた。図 4(c) のように学習する時に使う特定の長さのタイムステップの情報の中で前半は学習に使わず、後半だけ使う。ここで、前半部を Observation history、後半部を Updating

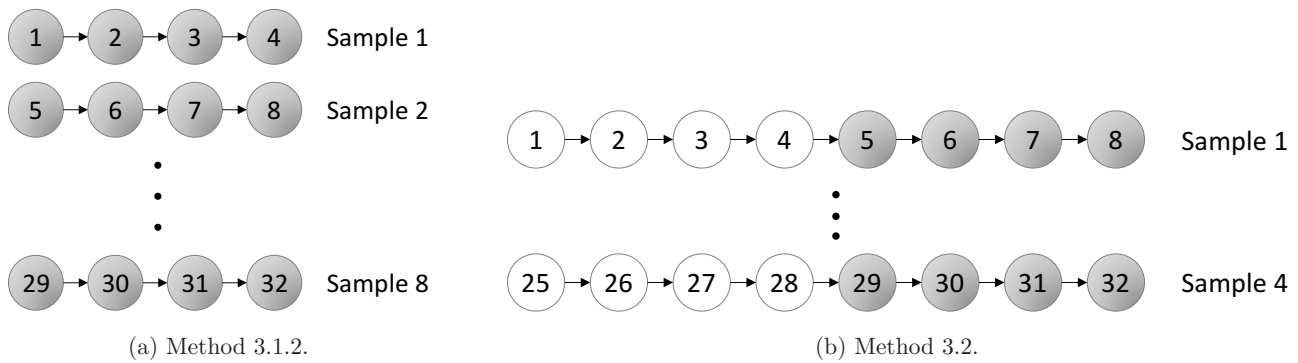


図 5: The difference in the number of sampled episodes during training between Bootstrapped Random Updates and Sequential Updates.

history と呼ぶ。すなわち、Observation history は時間情報的に意味がある hidden state を生成する役割をし、その hidden state を LSTM の初期状態として用いてエージェントを学習させる。

### 3.2.1 既存手法との比較

Sequential Updates は多様な場면을学習するという random sampling の観点では適していない。例えば、4 ステップの情報を用いて学習をしたい時、Bootstrapped Random Updates は 4 ステップの情報を用いることでできるが、Sequential Updates は時間情報的に意味がある hidden state を生成するための前半 4 ステップと、実際に学習に使うための後半 4 ステップを合わせて 8 ステップの情報が必要である。つまり、学習する時にリプレイメモリからある数の要素を抽出して学習するとすると、図 4(c) のように、エピソードの種類は Sequential Updates が Bootstrapped Random Updates の半分になる。条件を拡張して図 5 のように、4 ステップの情報を用いて学習し、リプレイメモリの要素 32 個を使おうとしよう。Bootstrapped Random Updates では、4 ステップで構成されている 8 個の異なるエピソードで学習することになる。一方、Sequential Updates では、有意義な hidden state を生成するための前半と学習に用いる後半を合わせて 8 ステップのステートを用いることになる。よって、8 ステップで構成されている 4 個の異なるエピソードで学習することになる。すなわち、同じ数のリプレイメモリの要素数を用いても学習に使える場面の多様性は半分になるため、random sampling の効果は減少する。一方、random sampling の効果を支えるために 4 ステップで構成されている 8 個の異なるエピソードで学習すると、学習に用いるステップは 2 ステップしかなくなるため、学習効率が減少する。

## 4. 提案手法

本稿では DRQN が長時間の情報をうまく学習することを目的として、図 4(b) のように学習サンプルの最初状態を FC に入力して LSTM の初期状態を生成する方法を提

案する。

エピソードの最初には以前のステップがないため、LSTM の初期状態を何らかの方法で設定する必要がある。初期状態を設定する方法としては、(1) ゼロベクトルや乱数のベクトルで設定する方法、(2) 初期状態をパラメータとして扱い、学習させる方法、の 2 つがある。ゼロベクトルや乱数のベクトルで初期状態を設定する方法が一般的に使われているが、文献 [5] では、実験によって初期状態を学習する必要があると報告されている。Bootstrapped Random Updates は LSTM の初期状態をゼロベクトルで初期化するため長時間の情報を学習することが難しいという欠点がある。本稿ではこれを解決するために、エージェントの学習時に FC 層を用いて LSTM の初期状態を生成する方法を提案する。これによって Bootstrapped Random Updates が長時間の情報を学習することが難しいという欠点を解決する。さらに、提案手法は random sampling の観点でも Bootstrapped Random Updates と同じ効果が期待できるため、Sequential Updates のような random sampling の効果の減少もない。

行動を選ぶために Q-値を出力するネットワークは、既存研究の DRQN と同じく図 3 のように構成されている。現在ステップのゲームフレームと 1 ステップ前の LSTM の状態を入力として扱い、Q-値を出力する。エージェントは epsilon greedy 政策によって行動を選ぶ。このネットワークと提案手法を適用した FC 層はステップ毎に学習を行い、学習時に提案手法によって LSTM の初期状態を生成する。

## 5. 実験

実験では、LSTM 層の初期状態をゼロベクトルにする Bootstrapped Random Updates、学習に用いるサンプルの前半は意味がある hidden state を生成することによって後半のみで学習を行う Sequential Updates、FC 層を用いて LSTM 層の初期状態を生成する提案手法の 3 つの手法を比較する。全ての実験において文献 [1] と同じような frame-

skipping 技術を適用した。具体的に、エージェントは 4 フレーム毎に状態を取得し、行動を選ぶ。実験は OpenAI gym [6] から提供されている Atari 2600 の一部のゲームに対して学習を行い、その性能を比較した。OpenAI gym が提供する環境を用いると、`gym.make("game name-v4")` によって環境を呼び出すことで簡単に 4 frame-skipping 技術を適用することができる。

## 5.1 Network Architecture

ネットワーク構造を説明するために、Fully Connected 層を `fc(number of units)`、Convolutional Neural Network 層を `conv(number of filters, size of filters, number of strides)`、LSTM 層を `lstm(number of hidden units)` として表記する。入力として DRQN は  $1 \times (84 \times 84 \times 1)$  の 1 つのフレームを用いる。

(1) 3 つの手法は交通のネットワーク構造を用いた。ネットワークは 4 つの CNN 層、1 つの LSTM 層、1 つの FC 層で構成されている: `conv(32, 8, 4) - conv(64, 4, 2) - conv(64, 3, 1) - conv(256, 7, 1) - lstm(256) - fc(number of actions)` で構成されている。

(2) 提案手法で追加した、循環神経網の初期入力を生成する FC 層は、bias を持っていない `fc(256)` を用いた。bias がない `fc(256)` を用いるために、`tensorflow` の `tf.Variable(tf.random_normal())` を用いた。

4 つの行動が選べるゲームに対して、Bootstrapped Random Updates, Sequential Updates のネットワークは 1,401,252 個のパラメータを、提案手法のネットワークは 1,532,324 個のパラメータを持つ。

## 5.2 Hyper-Parameters

全ての実験で discount factor は  $\gamma = 0.99$  に、Adam optimizer の learning rate を  $\alpha = 0.0001$  に設定した。Target network のアップデートは 10,000 ステップ毎に行った。ゲームのスタートポジションをランダムに設定するために各エピソードは 30 *no-op* actions によって始まった。50,000 ステップから 5,000,000 ステップまで学習を行った。Experience replay memory のサイズは 500,000 tuples であった。学習時に使うリプレイメモリの要素数は 32 に、batch size は 8 に設定した。1,000,000 ステップで  $\epsilon$  が 1 から 0.01 に減少する  $\epsilon$ -greedy を用いた。

## 5.3 Games

文献 [7] の実験結果を参考にして、DRQN の性能がよくないドメインとして Breakout, BeamRider を、DRQN の性能が良いドメインとして Seaquest, Space Invaders, Tutankham, Gopher を用いた。

### 5.3.1 Breakout

Breakout は画面の下段部にあるバーを用いて球を弾い

て画面の上段部にある壁を壊すゲームである。壁を壊すと球の速度が早くなる。

### 5.3.2 BeamRider

BeamRider は画面の上段部から近づいて来る敵を攻撃するゲームである。

### 5.3.3 Seaquest

Seaquest は 2 つのミッションがある。1 つ目は画面の左右から近づいて来る敵を攻撃することで、2 つ目は人を助けて画面上段部に連れて行くことである。

### 5.3.4 S. Invaders

S. Invaders は画面お下段部にいるエージェントが左右に動きながら画面の上段部にいる敵を攻撃するゲームである。

### 5.3.5 Tutankham

Tutankham は迷路の中でアイテムを獲得し、敵を攻撃しながら冒険するゲームである。

### 5.3.6 Gopher

Gopher は床の下にある gopher が穴を開けて上がってきて人参を盗むことを防ぐために、床の上にあるエージェントを左右に動きながら穴を埋めるゲームである。

## 5.4 実験結果

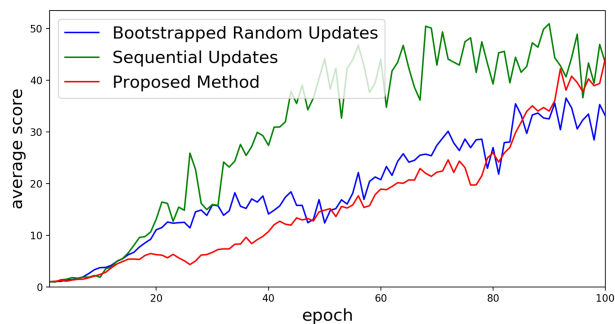
エージェントは 50,000 ステップを 1 エポックとしてその平均点数で性能を評価する。表 1 は 3 つのエージェントが 6 つのゲームに対して学習した結果、1 エポック毎の平均点数の最大値を示している。図 6 は Bootstrapped Random Updates, Sequential Updates, 提案手法の訓練の進捗状況を示している。実験結果、提案手法は 6 つのゲームの中で 4 つのゲームで 1 エポック毎の平均点数に対して最大点数を実現した。Breakout に関しては Sequential Updates の方が提案手法より点数が高かったが、図 6(a) を見ると、学習が進んだら提案手法の方が Sequential Updates の性能より良くなると思われる。BeamRider, Seaquest, Gopher は、図 6(b), 6(c), 6(f) のように、確実に改善ができた。しかし、S. Invaders, Tutankham は、図 6(d), 6(e) のように、他の手法とほとんど同じ様子を見せた。総合的に既存手法よりは良い性能ができると考えられる。

## 6. 終わりに

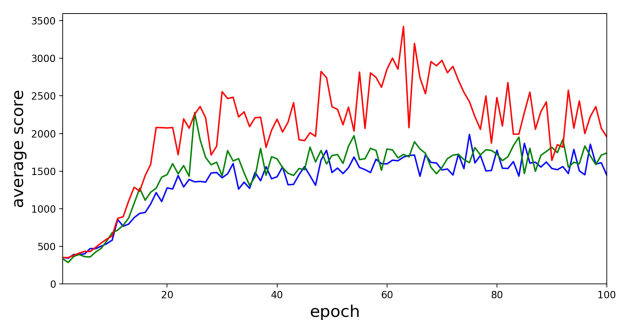
本研究では、DRQN が学習時に LSTM 層の初期状態をゼロベクトルで初期化するため発生する、長時間の情報を学習することが難しいという欠点を改善するために、全結合ネットワークを用いて LSTM 層の初期状態を生成する手法を提案した。提案手法を DRQN の学習手法の 1 つである Bootstrapped Random Updates, Sequential Updates と比較するために 6 つの Atari 2600 ゲームを用いて実験を行った。実験の結果、6 つの中で 4 つのゲームで提案手法が既存手法より良い性能を実現した。改善程度は提案手法のネットワーク構造によって異なるため、ネットワーク

表 1: The best average score per epoch for the three models on six Atari 2600 games. Bold face indicates the best score among the three models.

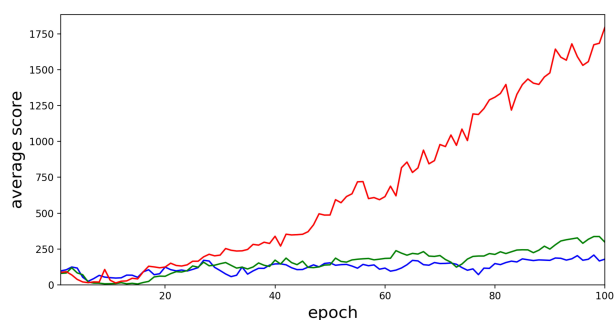
|                             | Breakout     | BeamRider      | Seaquest       | S. Invaders   | Tutankham    | Gopher        |
|-----------------------------|--------------|----------------|----------------|---------------|--------------|---------------|
| Bootstrapped Random Updates | 36.53        | 1987.75        | 208.08         | 316.88        | 47.00        | 880.00        |
| Sequential Updates          | <b>50.90</b> | 2276.73        | 337.07         | 340.58        | <b>80.00</b> | 771.43        |
| Proposed Method             | 44.17        | <b>3423.45</b> | <b>1793.89</b> | <b>344.43</b> | 67.00        | <b>966.67</b> |



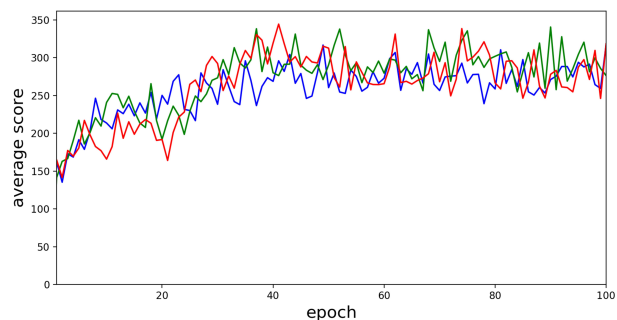
(a) Breakout



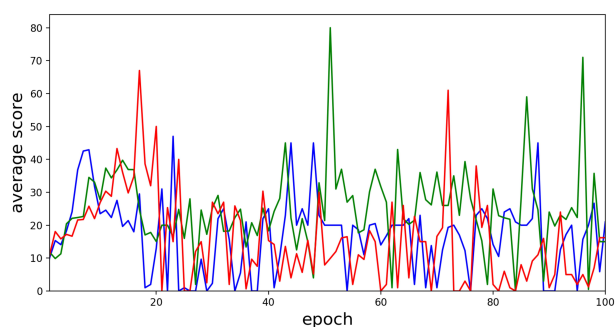
(b) BeamRider



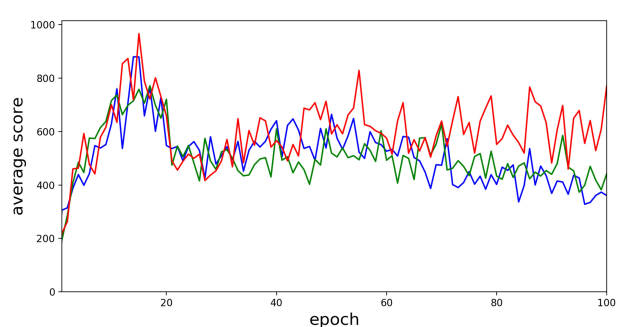
(c) Seaquest



(d) S. Invaders



(e) Tutankham



(f) Gopher

図 6: The average score per epoch of the three models on six Atari games. One epoch consists of 50,000 steps.

構造の調整によって本稿の結果より良い結果ができる構造を作ることができると思われる。

文献 [7] は DRQN に Attention Mechanism を適用した

強化学習アルゴリズムである Deep Attention Recurrent Q-Network (DARQN) を提案した。DARQN で適用された Attention Mechanism は、以前ステップの hidden state

と、現在ステップの特徴を入力とし、特徴の中で集中する必要があるところを学習する手法である。今後の課題として、本研究で提案した手法を DARQN に適用することによって性能の改善を確認する研究を考えている。

## 謝辞

この研究の一部は、JSPS 科研費 16H02927 と JST さきがけの支援を受けています。

## 参考文献

- [1] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529, 2015.
- [2] Matthew Hausknecht and Peter Stone. Deep recurrent q-learning for partially observable mdps. *CoRR, abs/1507.06527*, 2015.
- [3] Guillaume Lample and Devendra Singh Chaplot. Playing fps games with deep reinforcement learning. In *AAAI*, pages 2140–2146, 2017.
- [4] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- [5] Felix A Gers, Nicol N Schraudolph, and Jürgen Schmidhuber. Learning precise timing with lstm recurrent networks. *Journal of machine learning research*, 3(Aug):115–143, 2002.
- [6] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. Openai gym. *arXiv preprint arXiv:1606.01540*, 2016.
- [7] Ivan Sorokin, Alexey Seleznev, Mikhail Pavlov, Aleksandr Fedorov, and Anastasiia Ignateva. Deep attention recurrent q-network. *arXiv preprint arXiv:1512.01693*, 2015.